HCLTech Career Shaper Programme

III Year Industry Project, Academic Year 2025-26

PROJECT REPORT

Optimized Retail Inventory Management System

Demand forecasting, reorder alerts, supplier management and sales analytics

Field	Details
Project No.	21
Domain	Digital-Data-AI / AIML
Difficulty / Duration	Intermediate / 4 weeks (individual project)
Learner name	Apurv Sharma
Roll number	[FILL IN]
College / campus	[FILL IN]
Mentor name	[FILL IN]
Submission date	[FILL IN]

Contents

Contents 2
1. Summary 4
2. Introduction 5
2.1 Background 5
2.2 Problem statement and target user 5
2.3 Objectives 5
2.4 Success criteria 5
3. Data Description 6
3.1 Source and size 6
3.2 Generation assumptions and validation 6
3.3 Data dictionary 6
4. Data Preparation 7
5. Exploratory Data Analysis 8
5.1 Stock cover by category 8
5.2 Daily sales distribution and seasonality 8
5.3 Supplier lead times 9
5.4 Monthly demand trend 9
5.5 Store reorder rule vs real cover 10
6. Methodology and System Design 11
6.1 Architecture 11
6.2 Features and leakage control 11
6.3 Split and validation 12
6.4 Candidate models 12
6.5 Reorder logic 12
6.6 Supplier scorecard 12
7. Implementation 13
7.1 Tools and libraries 13
7.2 REST API 13
7.3 Dashboard 13
7.4 Testing 17
8. Results and Validation 18
8.1 Forecast accuracy 18
8.2 Error analysis 18

8.3 Alert validation 18
8.4 Limitations 19
9. Ethical Considerations 20
10. Conclusion and Future Work 21
11. References 22
Appendix A: Repository and Reproduction 23
Appendix B: AI Tool Disclosure 23

1. Summary

Retail stores lose sales when fast-moving products run out and lock up cash in products that sell slowly. The store manager has to decide what to reorder, when, and from which supplier. A fixed reorder level per product ignores both seasonal demand and the lead time of the supplier. This project builds a working prototype that replaces the fixed rule with a forecast-based one and presents the result to the manager through a REST API and a dashboard.

The inventory log supplied with the project brief (500 rows) was extended to 5,000 records covering 500 SKUs, 30 suppliers and all twelve months of 2024, using a reproducible script with documented assumptions. Supplier prices were converted into a masked Cost Index. Demand is forecast per SKU and season. Models were compared using a chronological split and time-series cross-validation, and the test period (1 October to 31 December 2024) was evaluated once.

The selected model, a seasonal naive forecast, reaches a test WAPE of 14.9% and a MAPE of 14.2%, against a WAPE of 31.0% for the naive forecast. The forecast-based reorder rule detects 99.5% of simulated stock-outs, compared with 24.1% for the store's fixed reorder level. All three success criteria set at the start of the project are met. The tuned machine-learning models did not outperform the simple seasonal methods on this data, which is discussed in Section 8.

2. Introduction

2.1 Background

Inventory decisions involve three recurring risks. A stock-out loses the sale and may lose the customer. Overstock ties up working capital and storage space. Supplier variability makes a reorder point that worked last month too low when delivery takes 21 days instead of 8. In Indian retail these risks are amplified by a strong seasonal pattern: a summer peak, a monsoon slowdown and a festive surge around Diwali.

2.2 Problem statement and target user

A retail store runs out of fast-selling products while slow-moving products tie up money. Store managers need to know what to reorder, when, and from which supplier. The target user is the store or inventory manager.

2.3 Objectives

- Build a working prototype with demand forecasting, reorder alerts, supplier management and sales analytics.
- Prepare the supplied dataset reproducibly and protect commercially sensitive supplier pricing.
- Compare forecasting approaches fairly, against baselines, without using the test period for model selection.
- Expose the logic through a REST API and an interactive dashboard.
- Validate the system against measurable criteria and document limitations.

2.4 Success criteria

Table 2.1: Success criteria set in Stage 1 and their outcome

Criterion	Target	Result	Met
Forecast error on the test period	WAPE below naive; MAPE 20% or lower	WAPE 14.9% vs 31.0%; MAPE 14.2%	Yes
Alerts detect simulated stock-outs	Recall 90% or higher	Recall 99.5%, precision 91.1%	Yes
Dashboard answers the business questions	What to reorder, overstocked category, slowest supplier	Reorder alerts tab, stock cover by category, supplier scorecard	Yes

3. Data Description

3.1 Source and size

The programme supplied dataset.csv with 500 rows and ten columns. All rows fall in the first days of January 2024 and are labelled Winter, so the file contains no seasonal history, and the category of a SKU differs between rows. To support the forecasting module the dataset was extended by script (prepare_data.py, fixed seed 42) to 5,000 records: 500 supplied rows plus 4,500 generated rows, 500 SKUs (SKU0001 to SKU0500), 30 suppliers (SUP01 to SUP30) and the dates 1 January to 31 December 2024. No external dataset was used.

3.2 Generation assumptions and validation

Table 3.1: Assumptions used to generate the additional records

Item	Assumption
SKU demand	Base demand uniform between 10 and 50 units per day for each SKU
Seasonal multipliers	Winter 1.00, Summer 1.20, Monsoon 0.95, Festive_Diwali 1.40 (1 October to 15 November); random noise with standard deviation 10%
Stock	About 18% of records at or below the reorder level
Suppliers	85% of orders from the primary supplier of the SKU; lead time distributed around a supplier mean, clipped to 3-21 days
Anomalies	150 generated rows (3%): 50 stock-outs, 50 demand surges (2.5-3.5 times normal), 50 supplier delays (22-35 days); the 500 supplied rows are unchanged
Cost	Supplier unit cost masked as a Cost Index (median 100, plus or minus 2% noise)

The generated data were validated against the assumptions: 5,000 rows, 500 SKUs, 30 suppliers, 12 months, no missing values, no duplicates and no negative stock or sales. Mean daily sales by season follow the intended order (Festive_Diwali 43.1, Summer 36.9, Winter 31.3, Monsoon 29.2 units per day).

3.3 Data dictionary

Table 3.2: Data dictionary (also submitted as data_dictionary.csv)

Column	Type	Unit	Description
date	datetime	-	Timestamp of the inventory log entry (YYYY-MM-DD HH:MM:SS); 2024-01-01 to 2024-12-31, 30 minutes apart within a day from 08:00
product_id	text	-	Pseudonymous SKU code SKU0001-SKU0500 (500 SKUs)
category	text	-	Product category: Grocery, Electronics, Apparel or HomeDecor
season	text	-	Retail season of the log date: Winter (16 Nov-Feb), Summer (Mar-Jun), Monsoon (Jul-Sep), Festive_Diwali (1 Oct-15 Nov)
stock_qty	integer	units	Units physically on hand at the log entry; 0 means a stock-out
reorder_level	integer	units	The store's fixed reorder point; the store flags an item when stock_qty <= reorder_level
daily_sales	integer	units/day	Units sold per day for the SKU on that log date (seasonal multipliers applied)
unit_cost	float	index points	MASKED supplier unit cost: relative Cost Index (dataset median = 100, +/-2% noise), not a currency amount
supplier_id	text	-	Pseudonymous supplier code SUP01-SUP30
lead_days	integer	days	Supplier lead time between placing and receiving an order

4. Data Preparation

The cleaning steps are implemented in prepare_data.py, which regenerates the processed dataset identically on every run. Table 4.1 lists each decision with its effect on the row count.

Table 4.1: Cleaning log

Step	Decision and reason	Rows affected
Parse dates	Converted to datetime; required for the chronological split	5,000
Missing values	Would be dropped; none present	0
Duplicates	Exact duplicates would be dropped; none present	0
Impossible values	Stock and sales clipped at 0, lead time at 1; none required	0
Valid extremes	Stock-outs, surges and delays kept as genuine operational events	150 (3.0%)
Sensitive data	unit_cost converted to a Cost Index because supplier pricing is commercially sensitive	5,000
Final size	5,000 rows, 500 SKUs, 30 suppliers	-

During modelling, the first record of each SKU has no earlier history and cannot be forecast. These 500 records are excluded in the modelling step only; the stored dataset keeps them.

The category of a SKU is inconsistent in the 500 supplied rows (252 of 500 SKUs appear with a single category, the others with two to four). Category analysis is therefore performed at record level.

5. Exploratory Data Analysis

The analysis answers five business questions. Each figure is followed by the insight it supports and the decision it informs. The figures are produced by eda.py from the processed dataset.

5.1 Stock cover by category

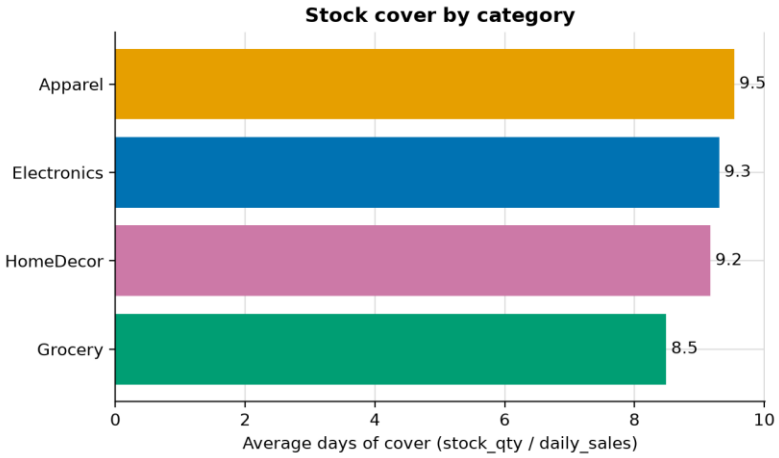


Figure 5.1: Stock cover by category

Insight. Apparel holds 9.5 days of cover on average against 8.5 for Grocery (1.12x).

Decision. Cover is similar across categories (spread under 25%), so manage replenishment per SKU rather than per category.

5.2 Daily sales distribution and seasonality

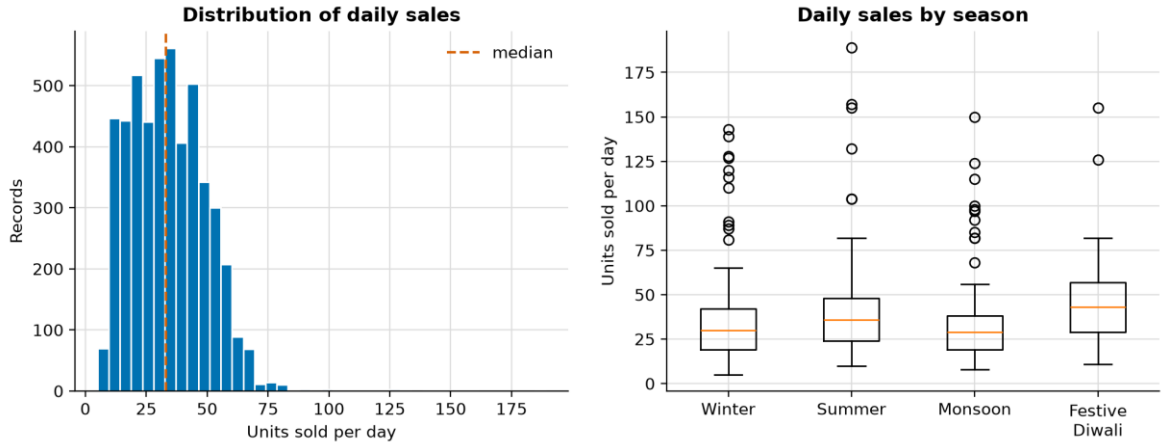


Figure 5.2: Daily sales distribution and seasonality

Insight. Median sales are 33 units/day with a long right tail (top 1% above 73). Festive_Diwali averages 43.1 units/day vs 31.3 in Winter (+38%).

Decision. Plan festive stock-ups ahead of 1 October and treat the extreme tail as demand surges, not as errors, when judging forecast accuracy.

5.3 Supplier lead times

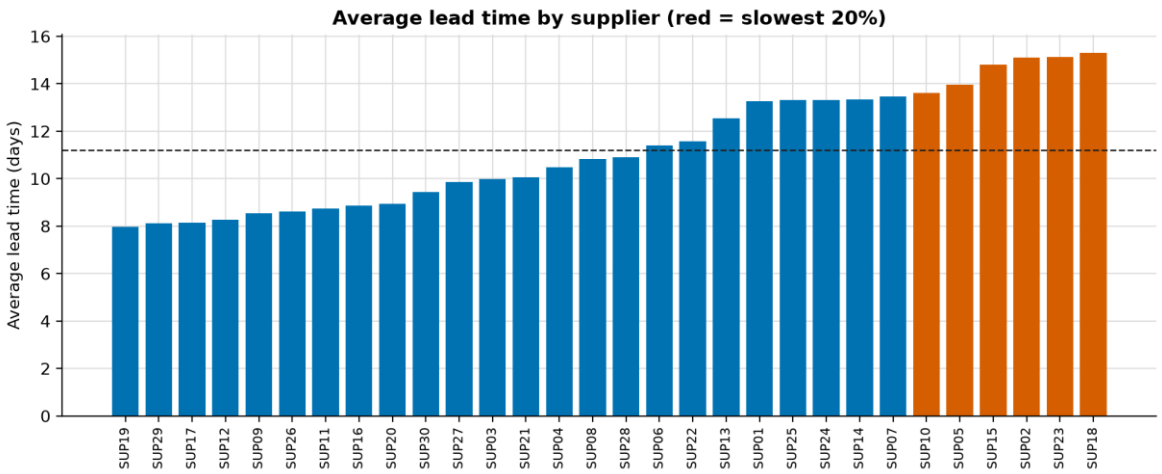


Figure 5.3: Supplier lead times

Insight. SUP18 is the slowest supplier at 15.3 days on average; SUP19 is the fastest at 8.0 days. 1.0% of all records exceed 21 days.

Decision. Move urgent orders away from SUP18 and open a delay review for suppliers in the red band.

5.4 Monthly demand trend

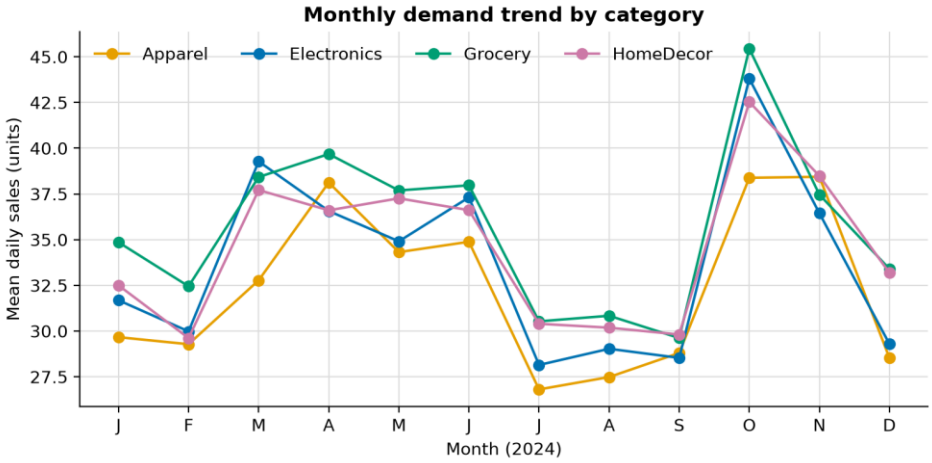


Figure 5.4: Monthly demand trend

Insight. Demand across categories peaks in month 10 (42.5 units/day) and is lowest in month 7 (29.0); all categories move together.

Decision. Seasonal calendar effects matter more than category differences, so forecasts must include season.

5.5 Store reorder rule vs real cover

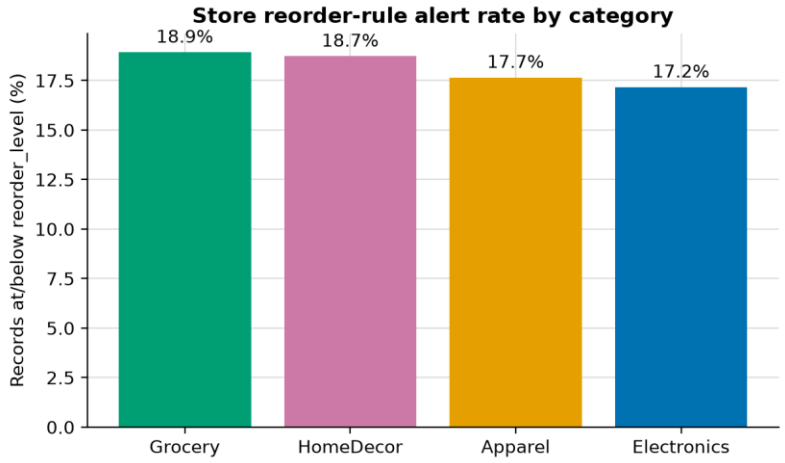


Figure 5.5: Store reorder rule vs real cover

Insight. Only 18.1% of records trigger the store's fixed reorder_level, yet 68.0% hold less stock than they will sell during the supplier lead time (days of cover < lead_days).

Decision. The fixed reorder_level under-warns. Use a forecast-based reorder point (demand over lead time x safety buffer); this is validated in Stage 10.

6. Methodology and System Design

6.1 Architecture

The system is a pipeline of five scripts that read and write well-defined files, followed by two interfaces. Each stage can be run on its own, and every figure and table in this report can be regenerated with the commands in Appendix A.

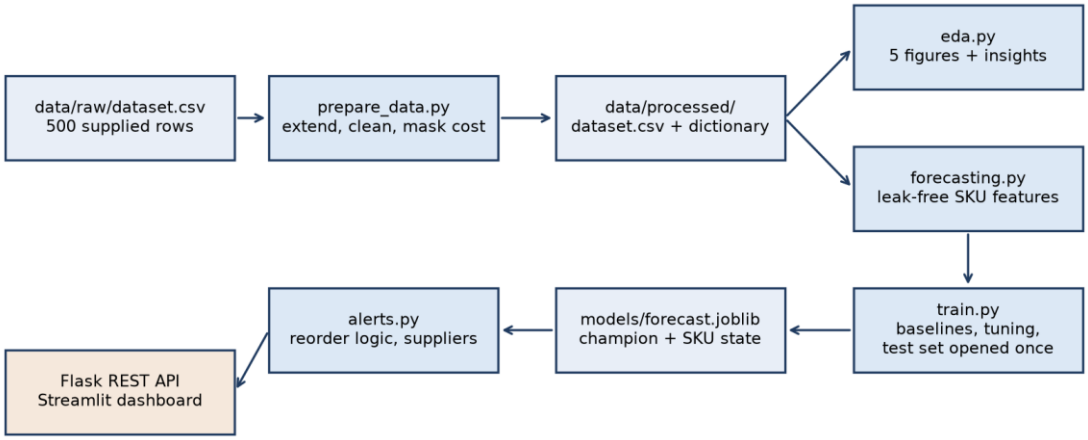


Figure 6.1: System architecture and data flow

Table 6.1: Modules

Module	Purpose	File
Data preparation	Extend, clean and mask the dataset	src/prepare_data.py
Exploratory analysis	Profile, figures and insights	src/eda.py
Forecasting	Leak-free features, metrics, forecast helper	src/forecasting.py
Model training	Baselines, tuning, evaluation, export	src/train.py
Alerts and suppliers	Reorder logic, supplier scorecard	src/alerts.py
REST API	Forecast, alert and supplier endpoints	api/app.py
Dashboard	Analytics, alerts, suppliers, forecast	app/dashboard.py

6.2 Features and leakage control

The target is daily_sales. Every feature of a record is computed from earlier records of the same SKU only: the history statistics are shifted by one record before any expanding mean is taken, so a record never contributes to its own features. The season is known from the calendar and is therefore a valid input.

Table 6.2: Engineered features

Feature	Definition
level	Expanding mean of de-seasonalised sales (sales divided by the season multiplier) of earlier records of the SKU
seas_level	level multiplied by the multiplier of the record's season (the seasonal level forecast)
seas_lag	Previous record of the SKU, de-seasonalised then re-seasonalised (the seasonal naive forecast)
lag_1, prior_mean	Previous sales of the SKU; plain mean of earlier sales (baseline forecasts)
mult, reorder_level, lead_days	Season multiplier and two supply-side attributes used by the learned models

6.3 Split and validation

Records before 1 October 2024 (3,286 usable records) form the training and validation period. Records from 1 October 2024 (1,214) form the test set and cover the festive season and the start of winter. The data are never shuffled. Each candidate is scored by four-fold TimeSeriesSplit cross-validation on the training period, hyper-parameters are tuned on the same folds, and the candidate with the lowest cross-validated WAPE is selected. The test set is evaluated once, after selection.

6.4 Candidate models

- Naive: the previous recorded sales of the SKU.
- SKU mean: the mean of earlier sales of the SKU, without seasonality.
- Seasonal naive: previous sales adjusted to the target season.
- Seasonal level rule: SKU level (de-seasonalised mean) adjusted to the target season.
- Linear regression, gradient boosting and random forest on the engineered features (grid search with TimeSeriesSplit).

6.5 Reorder logic

For each SKU the forecast daily demand is multiplied by the supplier lead time to give demand over the lead time. The reorder point adds a 20% safety buffer, an assumption chosen as a common starting value:

reorder point = forecast daily demand x lead days x 1.2

An alert is raised when stock is at or below the reorder point. The suggested order quantity is twice the reorder point minus stock, never below zero. The priority is Critical when stock is zero, High when days of cover are below half the lead time, and Medium otherwise.

6.6 Supplier scorecard

Suppliers are ranked by a risk score from 0 to 100 that combines 60% normalised average lead time, 25% delay rate (lead time above 21 days) and 15% stock-out rate. Tiers Low, Medium and High are assigned by tercile.

7. Implementation

7.1 Tools and libraries

Table 7.1: Technology stack (versions pinned in requirements.txt)

Layer	Technology
Language and data	Python 3.14, pandas 3.0, NumPy 2.4
Modelling	scikit-learn 1.8 (linear regression, gradient boosting, random forest, TimeSeriesSplit), joblib
Visualisation	matplotlib (EDA figures), Plotly (dashboard)
REST API	Flask 3.1
Dashboard	Streamlit 1.58
Testing	pytest 9.1 (8 tests: normal, boundary and invalid cases)

7.2 REST API

The Flask API validates its inputs and returns descriptive errors with HTTP status 400 for invalid values and 404 for an unknown SKU.

Table 7.2: API endpoints

Endpoint	Purpose
GET /api/health	Service status, selected model and test metrics
GET /api/forecast?product_id=&season=	Forecast daily units for a SKU in a season
GET /api/reorder-alerts?category=&priority=&season=	Current alerts with suggested order quantities
GET /api/suppliers?tier=	Supplier scorecard

Example: a request for SKU0001 in the Festive_Diwali season returns a forecast of 49.3 units per day. Sample responses from all endpoints are stored in reports/api_samples.json.

7.3 Dashboard

The Streamlit dashboard has four tabs and a sidebar with category, season and date filters. It runs with streamlit run app/dashboard.py.

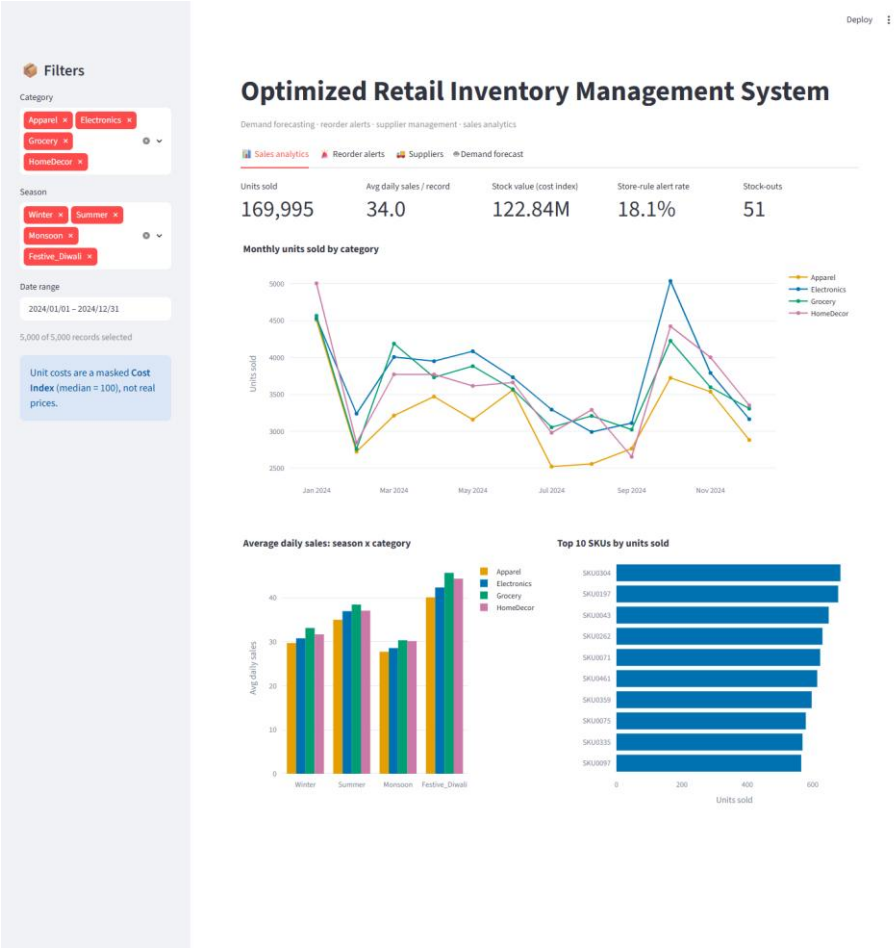


Figure 7.1: Sales analytics tab

The alerts tab evaluates the latest record of every SKU. With the forecast-based rule, 362 of 500 SKUs are flagged (5 Critical, 191 High, 166 Medium), compared with 93 under the store's fixed reorder level. The purchase list can be downloaded as CSV.

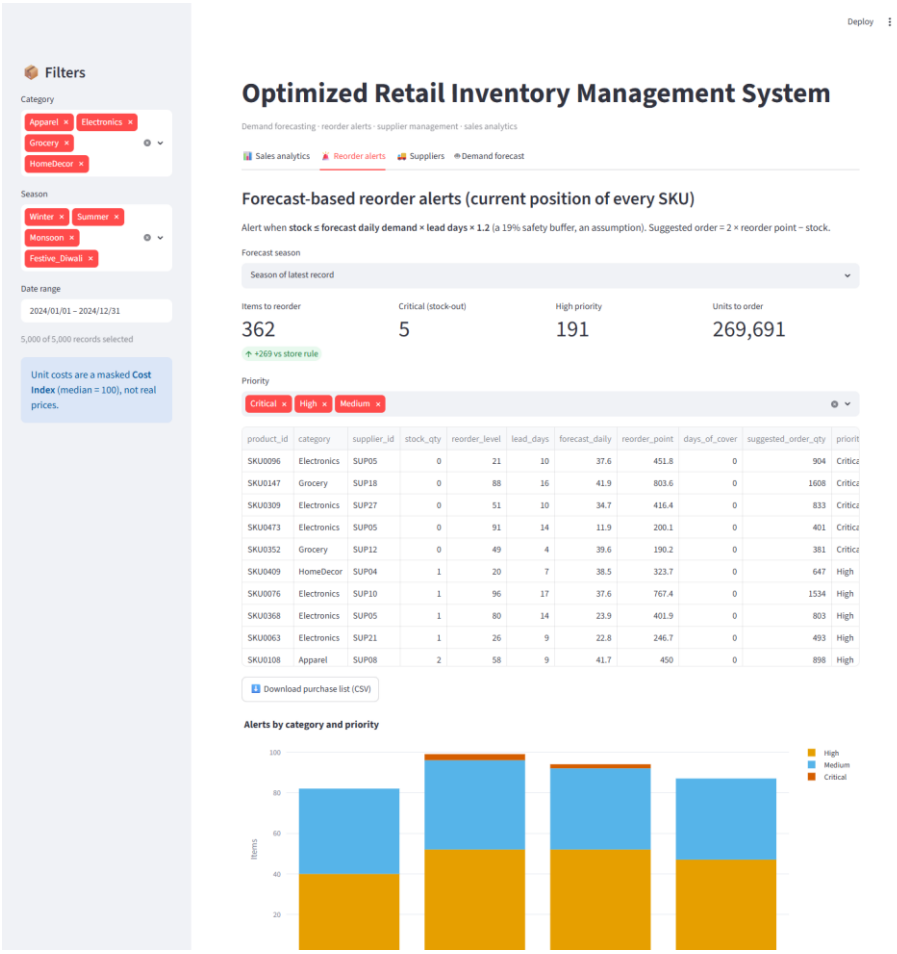


Figure 7.2: Reorder alerts tab

The supplier tab shows the scorecard. SUP23 has the highest risk score (80.8, average lead time 15.1 days); SUP18 has the longest average lead time (15.3 days). SUP09 has the lowest risk score.

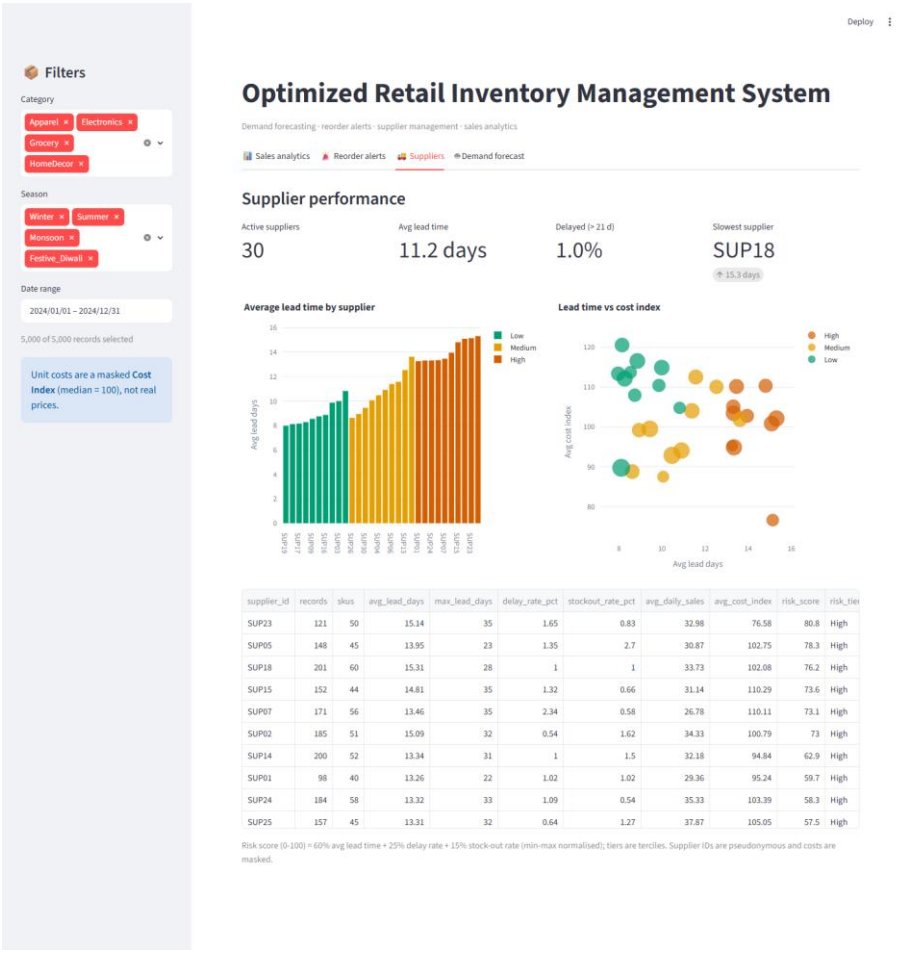


Figure 7.3: Supplier performance tab

The forecast tab applies the selected model to a chosen SKU, season, stock level and lead time, and shows the forecast, demand over the lead time, days of cover and a reorder recommendation. Selecting the festive season raises the forecast for SKU0001 from 35.2 to 49.3 units per day.

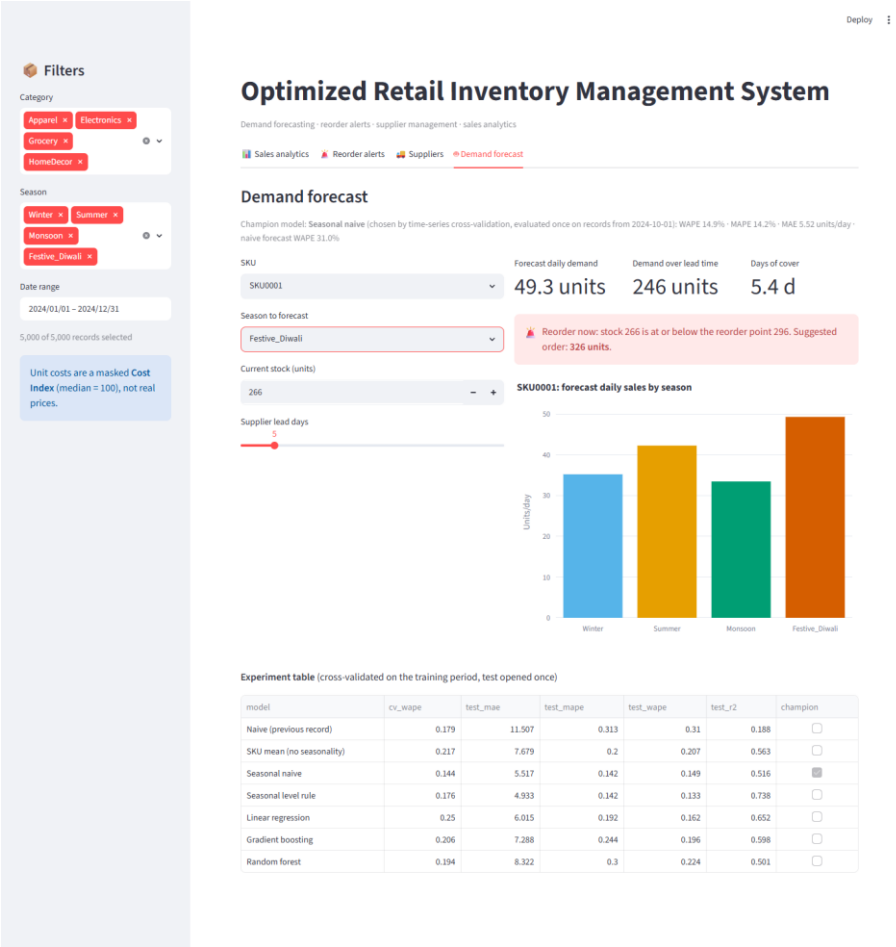


Figure 7.4: Demand forecast tab (Festive_Diwali selected)

7.4 Testing

Eight automated tests cover the API (valid requests, missing and invalid parameters, unknown SKU), the filters, the reorder boundary (stock equal to and just above the reorder point), the stock-out simulation and the absence of target leakage in the features. All tests pass (python -m pytest tests -q). The dashboard was also loaded headlessly with no exceptions.

8. Results and Validation

8.1 Forecast accuracy

Table 8.1: Experiment table (test period 1 Oct - 31 Dec 2024, 1,214 records)

Model	CV WAPE	Test MAE	Test RMSE	Test MAPE	Test WAPE	Test R2
Naive (previous record)	17.9%	11.51	15.87	31.3%	31.0%	0.19
SKU mean (no seasonality)	21.7%	7.68	11.64	20.0%	20.7%	0.56
Seasonal naive (selected)	14.4%	5.52	12.25	14.2%	14.9%	0.52
Seasonal level rule	17.6%	4.93	9.02	14.2%	13.3%	0.74
Linear regression	25.0%	6.02	10.39	19.2%	16.2%	0.65
Gradient boosting	20.6%	7.29	11.17	24.4%	19.6%	0.60
Random forest	19.4%	8.32	12.44	30.0%	22.4%	0.50

CV WAPE is the mean over four expanding-window folds of the training period and was the only basis for model selection. The seasonal naive forecast has the lowest cross-validated WAPE and was selected. The test results were computed once afterwards. The seasonal level rule has a lower test error than the selected model; this is reported but not acted upon, because choosing it now would use the test set for model selection.

The three machine-learning models did not outperform the seasonal rules. Each SKU has only about ten records, so the learned models have little history per product, and the festive uplift is not present in the training period for them to learn. The seasonal methods apply the documented season multipliers directly. A longer daily history per SKU would make the comparison fairer to the learned models.

8.2 Error analysis

Table 8.2: Error of the selected model by group

Group	Test WAPE
Season: Festive_Diwali	14.8%
Season: Winter	15.0%
Category: Apparel	15.2%
Category: Electronics	15.5%
Category: Grocery	13.6%
Category: HomeDecor	15.2%
Excluding the demand surges	13.5%

Error is similar across seasons and categories. 12 test records are demand surges (sales above 2.2 times the seasonal level); they cannot be predicted from the features and account for part of the remaining error (WAPE 13.5% without them).

8.3 Alert validation

The alerts were validated on the 1,214 test records against a simulated stock-out: a record is a stock-out when actual daily sales multiplied by the lead time exceed the stock on hand (a constant sales rate over the lead time is assumed).

Table 8.3: Alert validation (866 simulated stock-outs)

Rule	Alerts raised	Recall	Precision
Forecast-based reorder point	946	99.5%	91.1%

Store reorder_level (baseline)	210	24.1%	99.5%

The forecast-based rule finds almost all simulated stock-outs at the cost of more alerts, while the store's fixed rule misses about three quarters of them. The simulated stock-out rate is 71% because the generated stock levels are low relative to demand over the lead time, so recall is easier to reach than it would be with real stock data.

8.4 Limitations

- About 90% of the records are generated, so absolute figures illustrate the method and do not describe a real store.
- The seasonal multipliers are a planning assumption taken from the data generation, not learned from a previous year.
- With roughly ten records per SKU, simple seasonal rules are hard to beat; the conclusion may change with real daily sales history.
- The stock-out simulation assumes a constant sales rate over the lead time and uses generated stock levels.
- The 20% safety buffer is an assumption and was not optimised against a service-level target.
- The category of a SKU is inconsistent in the supplied rows.

9. Ethical Considerations

- Sensitive pricing: real supplier unit costs were replaced with a relative Cost Index before analysis. Real prices do not appear in the processed data, models, dashboard or this report, and the raw file is excluded from version control.
- Anonymisation: supplier and product identifiers are pseudonymous codes. No names, contact details or customer information are stored or processed.
- Generated data: 4,500 of 5,000 records are generated; the assumptions are documented and the report states that absolute metrics illustrate the method.
- Minimisation: only the fields needed for forecasting, alerts and supplier analysis are used.
- Fairness: the system makes no decisions about individuals.
- Transparency: weaker results are reported next to stronger ones, including the finding that the machine-learning models did not outperform simple seasonal methods.
- Secrets: the repository contains no passwords, API keys or personal data.

The signed Ethical Compliance Declaration is submitted as a separate document (docs/ethical_declaration.md).

10. Conclusion and Future Work

The project delivers a reproducible inventory management prototype that meets its three success criteria. A seasonal naive forecast reduces forecast error from 31.0% to 14.9% (WAPE) on a held-out festive and winter period, and a forecast-based reorder point detects 99.5% of simulated stock-outs where the store's fixed rule detects 24.1%. The REST API and dashboard make these results available to the store manager, and the supplier scorecard identifies where lead-time risk is concentrated.

Future work:

- Replace the generated history with real daily sales per SKU and re-run the model comparison.
- Learn seasonal factors from at least two years of data instead of assuming them.
- Add safety stock based on demand variability and a target service level, and economic order quantity.
- Add price and promotion effects to the forecast.
- Move storage from CSV files to a SQL database and automate purchase-order generation.

11. References

1. Pedregosa, F. et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830. https://jmlr.org/papers/v12/pedregosa11a.html
2. Hyndman, R. J. and Athanasopoulos, G. (2021). Forecasting: Principles and Practice, 3rd edition. OTexts. https://otexts.com/fpp3/
3. Friedman, J. H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. The Annals of Statistics, 29(5), 1189-1232. https://doi.org/10.1214/aos/1013203451
4. Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32. https://doi.org/10.1023/A:1010933404324
5. pandas, Flask, Streamlit and Plotly documentation: https://pandas.pydata.org, https://flask.palletsprojects.com, https://docs.streamlit.io, https://plotly.com/python/
6. HCLTech Career Shaper Programme (2025-26). Project Brief, Project No. 21: Optimized Retail Inventory Management System.
7. HCLTech Career Shaper Programme (2025-26). Industry Project Build Guide, Digital-Data-AI / AIML.
8. HCLTech Career Shaper Programme Guidelines (provided by the Programme Coordinator).

Appendix A: Repository and Reproduction

The complete source code is in the project repository. Folder layout:

Path	Content
data/raw, data/processed	Supplied dataset (not submitted), cleaned dataset and data dictionary
src/	prepare_data.py, eda.py, forecasting.py, train.py, alerts.py
api/, app/	Flask REST API and Streamlit dashboard
models/	forecast.joblib (selected model and SKU demand state)
reports/	Figures, screenshots, experiment table, results.json, this report
docs/	Charter, data source note, cleaning log, ethical declaration, demo video
tests/	Automated tests

Commands (Windows):

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
python src/prepare_data.py
python src/eda.py
python src/train.py
python -m pytest tests -q
python api/app.py
streamlit run app/dashboard.py
```

Appendix B: AI Tool Disclosure

In line with the programme's ethical guidelines, the use of AI tools in this project is disclosed as follows.

Item	Details
Tool	An AI coding assistant (large language model)
Used for	Drafting and debugging the Python scripts, reviewing the project against the programme build guide, and drafting the documentation and this report
Learner responsibility	The learner is responsible for the content of the final submission; all scripts were run and the reported results reproduced from the repository
Data	No real customer or supplier data was shared with the AI tool beyond the programme-supplied dataset.csv